Studio Nx

P1-10 회원 정보 화면 JS — 함수 설명 합본 (22~27)

2026-10-12

목차

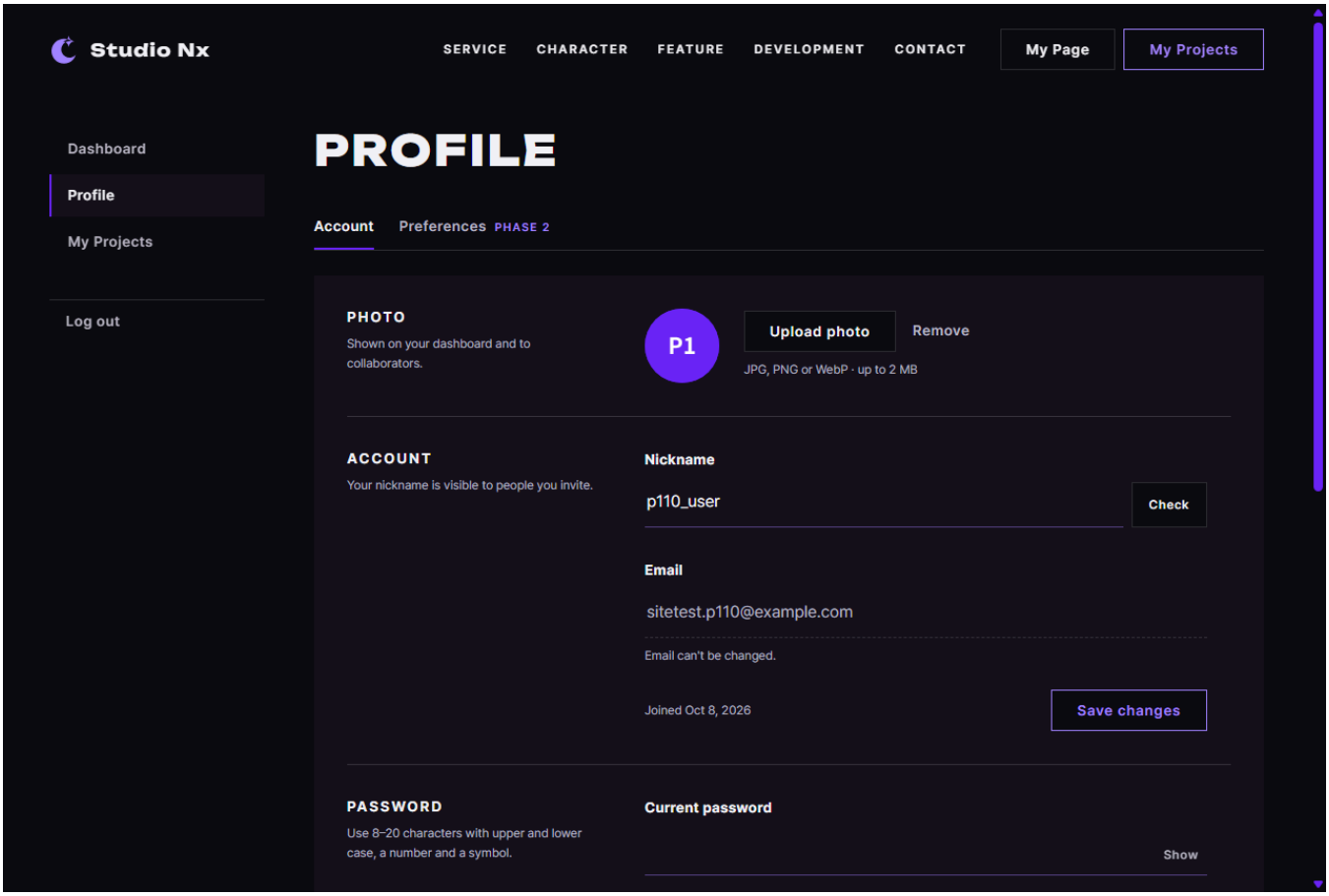
1. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/22_userinfo_loadMe.md
2. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/23_userinfo_initPhotoUpload.md
3. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/24_userinfo_initProfileForm.md
4. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/25_userinfo_initPasswordForm.md
5. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/26_userinfo_initWithdraw.md
6. ai-notes/javascript/P1-10 - 회원 정보 화면 JS/27_auth-ui_initLogoutButton.md

JS 22 — `loadMe()` (+ `renderAvatar()`)

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`
작업	P1-10 회원 정보 화면 JS: 내 정보 채우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	21 promptDialog() · 23 initPhotoUpload()

1. 하는 일

userInfo.html을 열면 서버에서 **내 정보**를 받아, 화면의 예시 값(`nightowl`, `nyxi@studionx.app`, `Joined Oct 1, 2026`, 보라색 `N`)을 실제 값으로 바꿉니다.



칸	채우는 값
Nickname 칸	`p110_user`
Email 칸 (읽기 전용)	`sitetest.p110@example.com`
가입일	`Joined Oct 8, 2026` 이며, 서버의 UTC 시각을 **이 컴퓨터의 시간대**로 바꿔서 보여 줍니다
프로필 동그라미	사진이 있으면 `<img>` 를, 없으면 닉네임의 첫 글자(대문자) `P` 를 보여 줍니다

불러오지 못하면 (2026-10-06 결정: "본 데이터 API 오류 → 오류 화면")

상태	옮겨 가는 곳
401 (로그인이 풀림)	`home.html#login?next=%2FuserInfo.html`
400, 403, 404, 429, 503, 504	`errors/<같은 번호>.html`
그 밖의 500 이상 (예: 502)	`errors/500.html`
서버에 닿지 못함 (네트워크 오류, 상태 0)	`errors/503.html`

로그인하지 않은 채로 열면 `loadMe`보다 먼저 `requireLogin()`(P1-02 07)이 로그인 주소로 보냅니다.

2. 언제 쓰나

순서	userinfo.js 아래쪽
①	`await includePartials()`로 헤더와 푸터를 넣습니다
②	`initAuthState()`, `initModalFallback()`, `initPasswordToggles()`
③	**`if (await requireLogin()) await loadMe();`** (템플릿의 TODO 두 줄을 이번에 바꿨습니다)
④	23~27번의 버튼 연결

`renderAvatar`는 23번(사진 올리기, 지우기)이 응답을 받은 뒤 다시 씁니다. 닉네임 칸의 `data-checked`는 24번(닉네임 저장)이 "확인을 통과한 값"으로 씁니다. 그래서 지금의 내 닉네임은 확인 없이 저장해도 됩니다.

3. 코드

import (파일 맨 위)

```
import { initAuthState, initPasswordToggles, setFieldState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
```

`loginUrl`과 `api` 줄을 이번에 추가했습니다.

상수와 도우미

```
/** 오류 화면이 있는 상태 코드 (errors/<상태>.html) */
const ERROR_PAGES = [400, 403, 404, 429, 500, 503, 504];

export function renderAvatar(me) {
  const avatar = document.getElementById('profile-avatar');
  if (!avatar) return;
  avatar.textContent = '';
  if (me.profile_image_url) {
    const img = document.createElement('img');
    img.src = me.profile_image_url;
    img.alt = ''; // 동그라미는 aria-hidden — 닉네임은 옆 칸에 글자로 있음
    avatar.appendChild(img);
  } else {
    avatar.textContent = me.nickname.charAt(0).toUpperCase();
  }
}
```

```
function formatDate(value) {
  const hasZone = /[zZ]$|[+-]\d\d:\d\d$/.test(value);
  const date = new Date(hasZone ? value : value + 'Z');
  return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' });
}
```

함수

```
export async function loadMe() {
  let me;
  try {
    me = await api.get('/users/me');
  } catch (err) {
    if (err.status === 401) {
      location.replace(loginUrl(location.pathname + location.search + location.hash));
    } else {
      let page = 500;
      if (ERROR_PAGES.indexOf(err.status) >= 0) page = err.status;
      else if (err.status === 0) page = 503;
      location.replace('errors/' + page + '.html');
    }
    return null;
  }

  const nickname = document.getElementById('profile-nickname');
  const nicknameField = document.getElementById('nickname-field');
  const email = document.getElementById('profile-email');
  const joined = document.getElementById('profile-joined');
  if (nickname) nickname.value = me.nickname;
  if (nicknameField) nicknameField.dataset.checked = me.nickname;
  if (email) email.value = me.email;
  if (joined) joined.textContent = 'Joined ' + formatDate(me.created_at);
  renderAvatar(me);
  return me;
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const page = ERROR_PAGES.includes(err.status) ? err.status : err.status === 0 ? 503 : 500;
…
nickname?.value = me.nickname;          // X — 이건 문법 오류 (?. 는 대입의 왼쪽에 못 씀)
…
const { nickname: name, profile_image_url: url } = me;
avatar.replaceChildren(url ? Object.assign(document.createElement('img'), { src: url, alt: '' }) : name.
at(0).toUpperCase());
```

최신 문법	실제 코드	뜻
`arr.includes(x)`	`arr.indexOf(x) >= 0`	목록에 있는지 확인합니다
삼항 연산자 이어 쓰기 `a ? b : c ? d : e`	`let page = 500; if … else if …`	여러 갈래로 나눕니다. 이어 쓰면 읽기 어렵습니다
`el?.value = …`	`if (el) el.value = …`	**대입에는 `?.`를 쓸 수 없습니다.** 최신 문법으로도 `if`가 필요합니다

최신 문법	실제 코드	뜻
`replaceChildren(…)`, `Object.assign(…)`	`textContent = ''` → `createElement` → `appendChild`	안을 비우고 새로 넣습니다
`str.at(0)`	`str.charAt(0)`	첫 글자입니다

문법 풀이

import 문법	뜻
`import { …, loginUrl } from '../core/auth.js'`	P1-02 04번으로, `home.html#login?next=…` 주소를 만듭니다
`import { api } from '../core/api.js'`	P1-02 02번으로, 토큰을 붙여 서버 API를 부르고, 실패하면 `ApiError(status, code, message)`를 던집니다

식	뜻
`try { … } catch (err) { … }`	`api.get`이 실패하면(`ApiError`) catch로 갑니다
`location.replace(주소)`	그 주소로 옮겨 가면서, 방문 기록에 지금 페이지를 **남기지 않습니다** (뒤로 가기로 오류 페이지나 깨진 페이지에 돌아오지 않게 하기 위해서입니다)
`/[zZ]$\|[+-]\d\d:\d\d$/.test(value)`	정규식입니다. 끝이 `Z`나 `+09:00` 같은 시간대 표시인지 확인합니다
`new Date('2026-10-07T01:12:30Z')`	끝에 `Z`가 있으면 UTC로 읽습니다. **없으면 이 컴퓨터의 시각으로 읽어서** 9시간이 어긋납니다
`toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' })`	`Oct 8, 2026` 모양으로 만듭니다 (사이트가 영어이기 때문입니다)
`dataset.checked = …`	`data-checked="…"` 속성입니다
`img.alt = ''`	그림 설명이 없다는 뜻으로, 꾸밈용 그림임을 나타냅니다

4. 흐름

1. `GET /users/me`를 보냅니다.
2. 실패하면 401일 때는 로그인 주소로, 아니면 상태에 맞는 오류 화면으로 옮기고 `null`을 돌려줍니다.
3. 성공하면 닉네임 칸, 이메일 칸, 닉네임 확인 표시, 가입일, 동그라미를 채우고 내 정보를 돌려줍니다.

5. 왜 이렇게

선택	이유
서버 날짜에 `Z`를 붙여서 읽음	서버(P1-08 36)는 UTC 시각을 시간대 표시 없이 보냅니다(`2026-10-07T23:30:00`). 그대로 `new Date`에 넣으면 한국 컴퓨터는 이를 한국 시각으로 읽어서 날짜가 하루 어긋날 수 있습니다. 이미 `Z`가 있으면 그대로 씁니다
오류 화면 표를 넓힘 (결정은 404, 403, 500 이상이었음)	400, 429, 503, 504 오류 화면도 이미 있으므로, 맞는 화면을 보여 주는 편이 더 정확합니다. 서버에 닿지 못하면(0) "잠시 이용 불가"를 뜻하는 503을 보여 줍니다
401은 오류 화면이 아니라 로그인으로 보냄	2026-10-06 결정입니다. 401은 "다시 로그인"이 맞습니다
`location.replace`	뒤로 가기를 눌러 깨진 회원 정보 화면으로 돌아오지 않게 합니다

선택	이유
동그라미 그리기를 따로 분리 (`renderAvatar`)	23번이 사진을 올리거나 지운 뒤 같은 방법으로 다시 그립니다
사진은 `<img>` 요소로 넣음 (`innerHTML`이 아님)	주소를 HTML 문자열로 끼워 넣지 않으므로 안전합니다
칸이 없으면 건너뜀 (`if (nickname)`)	화면을 고쳐서 칸이 빠지더라도 오류로 멈추지 않습니다

탈퇴 창의 닉네임(`#withdraw-nickname`)은 채우지 않았습니다. 템플릿에는 있었지만, 탈퇴는 FIX-01의 `promptDialog`로 바꾸기로 했기 때문에(26) 그 창의 HTML을 지울 예정입니다.

6. 시험 (실제 userInfo.html, 실제 서버, 헤드리스 Chrome)

시험 계정 `sitetest.p110@example.com`으로 가입하고 로그인한 뒤 페이지를 열었습니다 (시험 도구에 "열기 전에 로그인"과 "옮겨 간 뒤 확인" 기능을 더했습니다).

시험	확인한 것
페이지를 열면	닉네임 칸, 이메일 칸, `Joined Oct 8, 2026`(오늘), 첫 글자 `P`가 나오고, `data-checked`는 지금의 닉네임입니다
사진을 올린 뒤 다시 부름	`<img src="/api/v1/profile-images/{32자}.png" alt="">`가 들어가고, 그림이 실제로 뜨며(가로 60px), 내 정보를 돌려줍니다
사진을 지운 뒤 다시 부름	첫 글자로 돌아옵니다
`renderAvatar({ nickname: 'zed' })`	`Z`
실패 8가지 (API를 가짜 실패로 바꿔서)	404 → `/errors/404.html`, 403 → 403, 429 → 429, 502 → 500, 0 → 503, 503 → 503, 400 → 400, 401 → `/home.html#login?next=%2FuserInfo.html`이며, 모두 `null`을 돌려줍니다
로그인하지 않고 열기	`/home.html#login?next=%2FuserInfo.html`로 갑니다

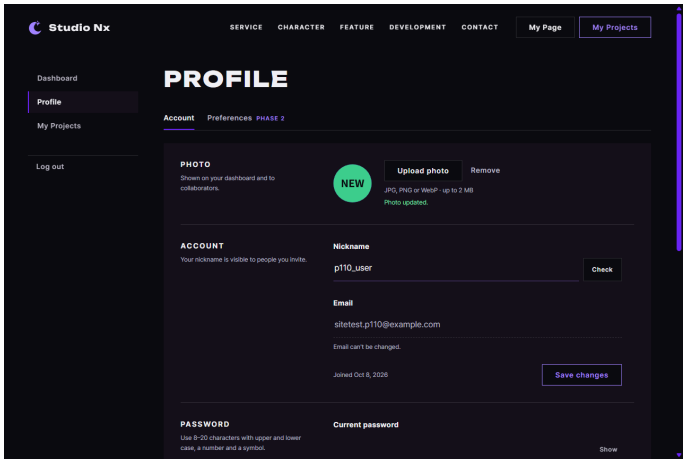
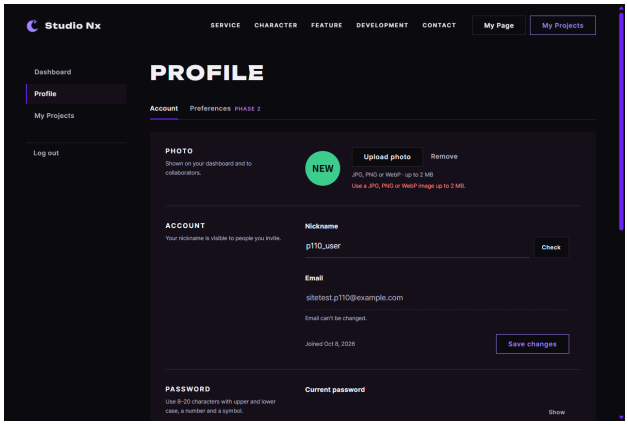
17개 모두 통과했습니다.

JS 23 — `initPhotoUpload()`

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`, `userInfo.html`(한 줄), `static/css/pages/userinfo.css`(세 줄)
작업	P1-10 회원 정보 화면 JS: 프로필 사진 올리기와 지우기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	22 `loadMe()` · 24 `initProfileForm()`

1. 하는 일

Upload photo로 고른 사진을 서버에 올리고, **Remove**로 지웁니다. 결과는 사진 아래에 한 줄로 알립니다.

올린 사진	거절 (GIF)
	

한 일	화면
PNG, JPG, WebP (2MB 이하)를 고름	바로 "Uploading..."이 나오고 버튼이 잠긴 뒤, 동그라미가 새 사진으로 바뀌고 초록 "Photo updated."가 나옵니다
GIF나 2MB가 넘는 파일을 고름	**보내지 않고** 빨강 "Use a JPG, PNG or WebP image up to 2 MB."를 보여 줍니다. 동그라미는 그대로입니다
확장자는 `.jpg`인데 내용은 사진이 아님	서버가 415로 응답하고 빨강 "Use a JPG, PNG or WebP image."가 나옵니다 (P1-08 38의 내용 검사)
Remove	닉네임 첫 글자로 돌아가고, 초록 "Photo removed."가 나옵니다
같은 파일을 다시 고름	다시 올라갑니다

2. 언제 쓰나

userInfo.html이 열릴 때 한 번 부릅니다 (`initPhotoUpload();`). 22번 `loadMe`가 처음 동그라미를 그리고, 이 함수는 사진을 바꾼 뒤 22번의 `renderAvatar`로 다시 그립니다.

쓰는 API	서버
`POST /api/v1/users/me/profile-image` (FormData, 칸 이름 `file`)	백엔드 42 → 38

쓰는 API	서버
`DELETE /api/v1/users/me/profile-image`	42 → 39

HTML과 CSS에 더한 것 (AI 화면 작업)

파일	내용
`userInfo.html`	사진 버튼, 안내, 문구를 감싼 `<div>`에 `id="photo-field"`를 붙였습니다. `setFieldState`가 상태 클래스를 붙일 곳입니다
`userinfo.css`	`#photo-field.is-ok`, `.is-error`일 때 `#photo-msg`를 초록, 빨강으로 바꾸고 위 여백을 줍니다. 이 문구는 폼 칸(`.field`) 밖에 있어서 폼 CSS의 색이 닿지 않았기 때문입니다

3. 코드

import (파일 맨 위: 22번과 같고, 추가한 것은 없습니다). 이 함수는 **`api`**, **`setFieldState`**, 22번의 **`renderAvatar`**, 템플릿의 **`PHOTO_TYPES`와 `PHOTO_MAX_BYTES`**를 씁니다.

함수

```
export function initPhotoUpload() {
  const input = document.getElementById('photo-input');
  const remove = document.getElementById('photo-remove');
  const field = document.getElementById('photo-field');
  if (!input) return;

  const setBusy = (busy) => {
    input.disabled = busy;
    if (remove) remove.disabled = busy;
  };

  input.addEventListener('change', async () => {
    const file = input.files && input.files[0];
    if (!file) return; // 고르다가 취소
    if (PHOTO_TYPES.indexOf(file.type) < 0 || file.size > PHOTO_MAX_BYTES) {
      setFieldState(field, 'error', 'Use a JPG, PNG or WebP image up to 2 MB.');
      input.value = '';
      return;
    }
    const form = new FormData();
    form.append('file', file);
    setBusy(true);
    setFieldState(field, null, 'Uploading…');
    try {
      renderAvatar(await api.post('/users/me/profile-image', form));
      setFieldState(field, 'ok', 'Photo updated.');
    } catch (err) {
      setFieldState(field, 'error', err.message);
    } finally {
      setBusy(false);
      input.value = '';
    }
  });

  if (remove) {
```

```
    remove.addEventListener('click', async () => {
      setBusy(true);
      try {
        renderAvatar(await api.delete('/users/me/profile-image'));
        setFieldState(field, 'ok', 'Photo removed.');
      } catch (err) {
        setFieldState(field, 'error', err.message);
      } finally {
        setBusy(false);
      }
    });
  }
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const file = input.files?.[0];
if (!PHOTO_TYPES.includes(file.type) || file.size > PHOTO_MAX_BYTES) { … }
…
remove?.addEventListener('click', async () => { … });
```

최신 문법	실제 코드	뜻
`input.files?.[0]`	`input.files && input.files[0]`	목록이 있을 때만 첫 번째를 꺼냅니다
`arr.includes(x)`	`arr.indexOf(x) >= 0` (여기서는 "없으면"이므로 `< 0`)	목록에 있는지 확인합니다
`remove?.addEventListener(…)`	`if (remove) { remove.addEventListener(…) }`	버튼이 있을 때만 연결합니다

문법 풀이

식	뜻
`'change'` (파일 칸)	파일을 고르고 확인했을 때 일어납니다. 같은 파일을 다시 고르면 값이 같아서 **일어나지 않으므로**, 끝날 때 `input.value = ''`로 비웁니다
`input.files[0]`, `file.type`, `file.size`	고른 파일, 브라우저가 알려 준 종류(`image/png`), 바이트 수입니다
`new FormData()`, `form.append('file', file)`	파일 올리기용 본문입니다. `api.js`는 FormData이면 종류 헤더를 비워 두므로, 브라우저가 경계(boundary)를 붙입니다
`renderAvatar(await api.post(…))`	서버 응답(내 정보)을 받자마자 그 사진 주소로 다시 그립니다
`try … catch … finally`	`finally`는 성공해도 실패해도 실행됩니다. 잠금 풀기를 한 곳에서 할 수 있습니다
`<label for="photo-input">` (HTML)	진짜 파일 칸은 숨겨 두고, 버튼 모양의 이름표를 누르면 파일 고르기 창이 열립니다. 칸이 `disabled`이면 이름표를 눌러도 열리지 않습니다

4. 흐름

파일을 골랐을 때: 취소했으면 끝냅니다. 종류나 크기가 틀리면 빨간 문구를 보여 주고 끝냅니다. 맞으면 "Uploading…"을 보여 주고 버튼을 잠근 뒤 올립니다. 성공하면 새 사진과 초록 문구를, 실패하면 서버 문구를 빨

강으로 보여 줍니다. 마지막에 잠금을 풀고 칸을 비웁니다. **Remove를 눌렀을 때:** 버튼을 잠그고 사진을 지웁니다. 성공하면 첫 글자와 초록 문구를, 실패하면 빨간 문구를 보여 주고 잠금을 풉니다.

5. 왜 이렇게

선택	이유
화면에서 먼저 종류와 크기를 검사	10MB 사진을 다 올린 뒤에야 거절당하지 않게 합니다. 서버도 같은 기준으로 다시 검사합니다 (브라우저가 알려 준 종류는 믿을 수 없기 때문입니다, 38)
화면 문구는 하나 ("… up to 2 MB.")	종류가 틀리든 크기가 틀리든 고치는 방법이 같으므로, 조건을 한 번에 알려 줍니다
서버가 거절하면 서버 문구를 그대로 보여 줌	화면 검사를 통과했는데 내용이 사진이 아닌 경우는 서버만 알 수 있습니다
"Uploading…"과 버튼 잠금	느린 망에서 두 번 올리거나, 올리는 중에 지우는 일을 막습니다
성공 문구 (템플릿에는 없었음)	동그라미만 바뀌면 화면 읽기 프로그램 사용자는 알 수 없습니다. `#photo-msg`는 `aria-live` 칸입니다
사진이 없을 때도 Remove를 둠	서버가 200을 줍니다(39). 숨기면 사진 유무에 따라 화면이 흔들립니다

6. 시험 (실제 userInfo.html, 실제 서버, 헤드리스 Chrome, 9개)

사진은 페이지 안에서 canvas로 만들고, `DataTransfer`로 파일 칸에 넣은 뒤 `change`를 보냈습니다 (사람이 고른 것과 같은 이벤트입니다).

시험	확인한 것
PNG 올리기	보내는 중에 "Uploading…"이 나오고 두 버튼이 잠깁니다. 이어서 동그라미가 `<img src="/api/v1/profile-images/{32자}.png">`로 바뀌고 "Photo updated."가 초록(`rgb(127, 224, 168)`)으로 나오며, 잠금이 풀리고 칸이 비워집니다
GIF, 2MB+1바이트	빨강(`rgb(255, 138, 138)`) 문구가 나오고, **요청은 0번**이며, 동그라미는 앞의 사진 그대로입니다
`.jpg`인데 내용이 HTML	요청을 1번 보내고, 서버의 415 문구 "Use a JPG, PNG or WebP image."가 나옵니다
Remove	첫 글자 `P`와 "Photo removed."가 나옵니다
같은 파일을 다시 고름	다시 올라갑니다

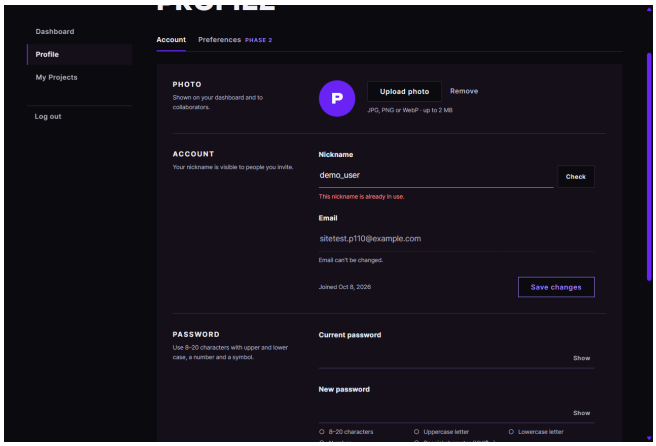
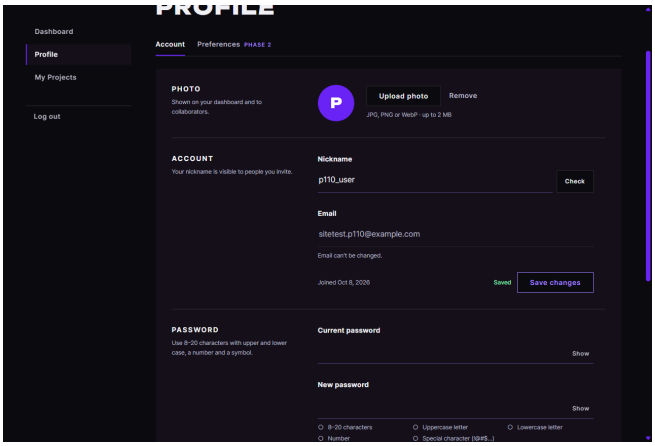
9개 모두 통과했습니다. 22번 시험(8개)도 다시 돌려서 통과했습니다.

JS 24 — `initProfileForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`, `userInfo.html` (Saved 자리), `static/css/pages/userinfo.css`
작업	P1-10 회원 정보 화면 JS: 닉네임 중복 확인과 저장 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	23 `initPhotoUpload()` · 25 `initPasswordForm()`

1. 하는 일

닉네임을 바꿀 때 **가입 화면과 같은 방식**으로 중복을 확인하고, **Save changes**로 저장합니다.

이미 쓰는 닉네임	저장됨
	

입력 (지금 닉네임은 `p110_user`)	칸 아래
`demo_user` (다른 사람이 씀)	입력을 멈추고 0.5초가 지나면 빨강 "This nickname is already in use."가 나옵니다
`a`	자동으로 확인하지 않고, Check를 누르면 "Use 2–20 characters."가 나옵니다
`P110_User` (대소문자만 다름)	**서버에 묻지 않고** "✓ Available"을 보여 줍니다. 내 닉네임이기 때문입니다
`p110_user` (지금 그대로)	아무 글도 나오지 않습니다
`zeta_p110` (새 이름)	"✓ Available"
탈퇴한 지 3일이 안 된 닉네임	"You can use this nickname after Oct 9."

Save changes	결과
확인을 통과한 값	저장한 뒤 버튼 옆에 초록 **Saved**가 나오고(3초 뒤 사라짐), 사진이 없으면 동그라미의 첫 글자도 새로 바뀝니다(`Z`)
확인하기 전에 바로 누름	"Click Check to confirm this nickname is available."이 나오고, 저장하지 않습니다
사용 중으로 나온 값	그 문구가 그대로 남고, 저장하지 않습니다
확인한 뒤 다른 사람이 먼저 가져감 (서버 409)	칸에 "This nickname is already in use."가 나옵니다
그 밖의 서버 오류	폼 아래의 `#profile-error`에 보여 줍니다

2. 언제 쓰나

userInfo.html이 열릴 때 22번 `loadMe` **다음에** 한 번 부릅니다. 22번이 닉네임 칸에 `data-checked = 지금 닉네임`을 넣어 두므로, 이 함수는 그 값을 "지금 내 닉네임"으로 삼습니다.

쓰는 API	서버
`GET /api/v1/auth/check-nickname?nickname=` (로그인 없이)	백엔드 24이며, 가입 화면과 같은 API입니다
`PATCH /api/v1/users/me`	백엔드 41 → 37

HTML과 CSS에 더한 것: Save changes 버튼 옆에 `<span id="profile-saved" role="status">`와 이를 감싸는 `<div class="ui-save__end">`를 넣었습니다. 글자는 초록이고, 비어 있으면 숨기며, 휴대폰에서는 버튼 위 가운데에 놓입니다.

3. 코드

import (파일 맨 위: 22번과 같고, 추가한 것은 없습니다). **`api`, `setFieldState`, `renderAvatar`(22)**를 씁니다.

상수

```
/** 닉네임 길이 (서버 Nickname 타입 · 가입 화면과 같음) */
const NICKNAME_MIN = 2;
const NICKNAME_MAX = 20;
```

함수: 준비와 확인

```
export function initProfileForm() {
  const form = document.getElementById('profile-form');
  const input = document.getElementById('profile-nickname');
  const field = document.getElementById('nickname-field');
  const checkButton = document.querySelector('#profile-form [data-check="nickname"]');
  const error = document.getElementById('profile-error');
  const saved = document.getElementById('profile-saved');
  const submit = form ? form.querySelector('button[type="submit"]') : null;
  if (!form || !input || !field) return;

  let current = field.dataset.checked || input.value.trim(); // 지금 내 닉네임 (22 가 넣어 둠)
  let timer = null;
  let savedTimer = null;
  const isValid = (value) => value.length >= NICKNAME_MIN && value.length <= NICKNAME_MAX;

  const check = async () => {
    clearTimeout(timer);
    delete field.dataset.checked; // 이번 확인을 통과해야 다시 남김
    const value = input.value.trim();
    if (!isValid(value)) {
      setFieldState(field, 'error', 'Use 2–20 characters.');
      return;
    }
    if (value === current) {
      field.dataset.checked = value;
      setFieldState(field, null); // 내 닉네임 그대로 — 안내 글로
      return;
```

```
    }
    if (value.toLowerCase() === current.toLowerCase()) {
      field.dataset.checked = value;
      setFieldState(field, 'ok', '✓ Available');
      return;
    }
    setFieldState(field, null, 'Checking…');
    let result;
    try {
      result = await api.get('/auth/check-nickname?nickname=' + encodeURIComponent(value), { auth: false
});
    } catch (err) {
      if (input.value.trim() !== value) return; // 그사이 입력이 바뀜 — 옛 응답은 버림
      setFieldState(field, 'error', err.status === 422 ? 'Use 2–20 characters.' : err.message);
      return;
    }
    if (input.value.trim() !== value) return;
    if (result.available) {
      field.dataset.checked = value;
      setFieldState(field, 'ok', '✓ Available');
    } else if (result.reason === 'REJOIN_TOO_SOON') {
      const day = new Date(result.available_at).toLocaleDateString('en-US', { month: 'short', day: 'nume
ric' });
      setFieldState(field, 'error', `You can use this nickname after ${day}.`);
    } else {
      setFieldState(field, 'error', 'This nickname is already in use.');
    }
  };

  input.addEventListener('input', () => {
    delete field.dataset.checked;
    setFieldState(field, null);
    if (saved) saved.textContent = '';
    clearTimeout(timer);
    if (isValid(input.value.trim())) timer = setTimeout(check, 500);
  });
  if (checkButton) checkButton.addEventListener('click', check);
```

함수: 저장

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    clearTimeout(timer);
    if (error) error.hidden = true;
    const value = input.value.trim();
    if (!isValid(value)) {
      setFieldState(field, 'error', 'Use 2–20 characters.');
      input.focus();
      return;
    }
    if (field.dataset.checked !== value) {
      if (!field.classList.contains('is-error')) {
        setFieldState(field, 'error', 'Click Check to confirm this nickname is available.');
      }
      input.focus();
      return;
```

```
    }
    if (submit) submit.disabled = true;
    try {
      const me = await api.patch('/users/me', { nickname: value });
      current = me.nickname;
      input.value = me.nickname;
      field.dataset.checked = me.nickname;
      setFieldState(field, null);
      renderAvatar(me); // 사진이 없으면 첫 글자가 바뀔 수 있음
      if (saved) {
        saved.textContent = 'Saved';
        clearTimeout(savedTimer);
        savedTimer = setTimeout(() => { saved.textContent = ''; }, 3000);
      }
    } catch (err) {
      if (err.status === 409 || err.status === 422) {
        delete field.dataset.checked;
        setFieldState(field, 'error', err.status === 422 ? 'Use 2–20 characters.' : err.message);
        input.focus();
      } else if (error) {
        error.textContent = err.message || 'Something went wrong. Please try again.';
        error.hidden = false;
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const submit = form?.querySelector('button[type="submit"]');
let current = field.dataset.checked ?? input.value.trim();
…
checkButton?.addEventListener('click', check);
error && (error.hidden = true);
…
const { nickname } = await api.patch('/users/me', { nickname: value });
```

최신 문법	실제 코드	뜻
`form?.querySelector(…)`	`form ? form.querySelector(…) : null`	폼이 있을 때만 찾습니다
`a ?? b`	`a || b`	없으면 b를 씁니다 (여기서는 빈 닉네임이 없어서 `||`와 결과가 같습니다)
`checkButton?.addEventListener(…)`	`if (checkButton) checkButton.addEventListener(…)`	있을 때만 연결합니다
`error && (error.hidden = true)`	`if (error) error.hidden = true`	`&&`로 줄여 쓰는 습관입니다. 읽기 어렵습니다
`const { nickname } = …`	`const me = …` + `me.nickname`	응답에서 칸을 꺼냅니다

문법 풀이

식	뜻
`let current = …`	저장에 성공하면 값이 바뀌므로 `const`가 아니라 `let`으로 선언합니다
`value.toLowerCase() === current.toLowerCase()`	대소문자만 다른지 확인합니다
`setTimeout(check, 500)`, `clearTimeout(timer)`	입력을 멈춘 뒤 0.5초가 지나면 확인합니다 (가입 화면 15번과 같습니다)
`encodeURIComponent(value)`	주소에 넣을 수 있게 바꿉니다 (`+`, `&` 등)
`delete field.dataset.checked`	`data-checked` 속성을 지웁니다
`form.requestSubmit()` (시험에서 씀)	사람이 Save를 누른 것처럼 `submit`을 일으킵니다

4. 흐름

입력할 때: 확인 표시, 문구, Saved를 지웁니다. 길이가 맞으면 0.5초 뒤에 확인하도록 예약합니다. **확인할 때:** 길이를 검사하고, 지금 닉네임 그대로면 끝냅니다. 대소문자만 다르면 ✓를 보여 줍니다. 아니면 서버에 묻고, 늦게 온 응답은 버리며, 결과에 따라 사용 가능, 탈퇴 3일, 사용 중 문구를 보여 줍니다. **저장할 때:** 길이와 확인 통과 여부를 검사한 뒤 버튼을 잠그고 PATCH를 보냅니다. 성공하면 Saved와 첫 글자를 바꾸고, 409와 422는 칸에, 그 밖의 오류는 폼 아래에 보여 준 뒤 잠금을 풉니다.

5. 왜 이렇게

선택	이유
가입 화면(15, 17)과 같은 규칙, 문구, 0.5초	같은 닉네임 규칙을 두 화면에서 다르게 보여 주면 헷갈립니다
대소문자만 바꾸면 서버에 묻지 않음	확인 API는 대소문자를 구분하지 않고 비교하므로 **내 닉네임을 "사용 중"**으로 답합니다. 서버의 저장(37)은 이것을 허용하므로 화면도 맞춰야 합니다
지금 닉네임은 22번이 남긴 `data-checked`에서 읽음	따로 저장하지 않고 22번과 이어서 씁니다. 저장에 성공하면 `current`를 바꿉니다
확인을 통과해야 저장	템플릿의 요구 사항이며, 가입 화면과 같습니다. 대부분 0.5초 자동 확인이 끝나 있으므로 따로 누를 일은 적습니다
409를 칸에 표시	확인과 저장 사이에 다른 사람이 먼저 가져간 경우, 고칠 곳을 바로 알려 줍니다
Saved는 3초 뒤 사라지고, 입력하면 바로 지움	오래 남아 있으면 "지금 칸의 값이 저장되었다"고 오해할 수 있습니다
탈퇴 3일 문구를 "can use … after"로 바꿈	가입 화면의 "sign up again after"는 이 화면에 맞지 않습니다 (P1-08 37과 같은 이유)

6. 시험 (실제 userInfo.html, 실제 서버, 15개)

시험	확인한 것
처음	지금 닉네임과 `data-checked`가 있고, 문구는 없습니다
그대로 Save	Saved가 나오고, PATCH를 1번 보냅니다
`demo_user`	입력하자 Saved와 확인 표시가 지워지고, 0.3초에는 요청이 0번이며, 0.5초가 지나면 "already in use"가 나옵니다. Save를 눌러도 PATCH는 0번입니다

시험	확인한 것
`a`	자동 확인은 0번이고, Check를 누르면 "Use 2–20 characters."가 나옵니다 (요청 0번)
`P110_User`	요청 0번으로 ✓가 나오고, 저장하면 서버에도 `P110_User`로 저장됩니다
`zeta_p110`	✓가 나오고, 저장하면 첫 글자가 `Z`로 바뀌며, 3초 뒤 Saved가 사라집니다
확인하기 전에 바로 Save	"Click Check…"가 나오고, PATCH는 0번입니다
서버 409 (확인 표시를 꾸며서 다른 사람의 닉네임으로 저장)	칸에 "already in use"가 나오고, 확인 표시가 지워지며, 버튼이 다시 켜집니다
되돌리기	서버에 `p110_user`로 돌려 놓았습니다

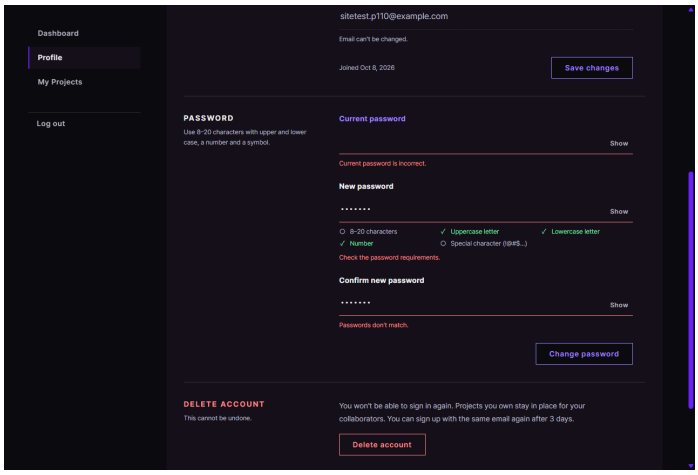
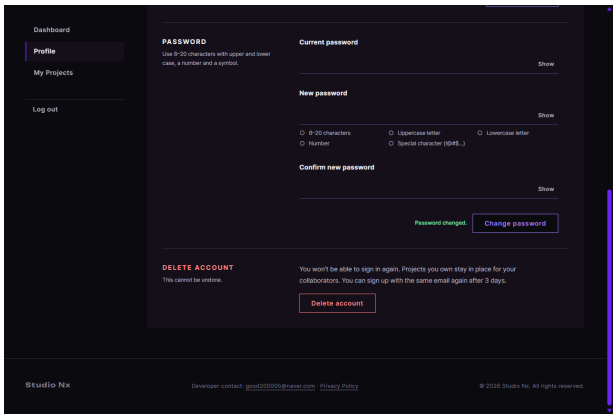
15개 모두 통과했습니다. 22번(8개)과 23번(9개) 시험도 다시 돌려서 통과했습니다.

JS 25 — `initPasswordForm()` (+ `core/password.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`, 새 파일 `static/js/core/password.js`, `pages/join.js`(import 한 줄), `userInfo.html`(문구 자리)
작업	P1-10 회원 정보 화면 JS: 비밀번호 변경 (PRD 01 FR-107)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	24 `initProfileForm()` · 26 `initWithdraw()`

1. 하는 일

회원 정보 화면의 **Password** 칸 세 개(지금, 새, 확인)를 가입 화면과 같은 방식으로 검사하고, **Change password**로 비밀번호를 바꿉니다.

고칠 곳 표시	바뀜
	

한 일	화면
새 비밀번호에 `Studio1` 입력	조건 목록에서 대문자, 소문자, 숫자가 ✓가 됩니다 (입력 중에는 빨간 글이 없습니다). 칸을 벗어나면 "Check the password requirements."가 나옵니다
새 비밀번호에 한글 입력	바로 "Use only English letters, numbers and symbols."가 나옵니다
확인 칸이 다름	같은 길이까지 쳤거나 칸을 벗어나면 "Passwords don't match."가 나오고, 같아지면 바로 지웁니다
빈 채로 Change password	세 칸에 문구가 나오고, 커서는 지금 비밀번호 칸에 두며, **보내지 않습니다**
지금 비밀번호가 틀림 (서버 400)	"Current password is incorrect."가 나오고, 그 칸을 비운 뒤 커서를 둡니다. **로그아웃되지는 않습니다**
새 비밀번호가 지금과 같음 (서버 422)	새 비밀번호 칸에 "Use a different password."가 나옵니다 (P1-09 사용자 결정)
바뀜 (204)	세 칸을 비우고, 조건을 ○로 되돌리며, 버튼 옆에 초록 "Password changed."가 나옵니다 (5초)

2. 언제 쓰나

userInfo.html이 열릴 때 한 번 부릅니다. 서버는 백엔드 47 `PUT /users/me/password`입니다.

규칙 파일을 나눔: `core/password.js` (템플릿 권장)

전	지금
`PASSWORD_RULES`와 `PASSWORD_ALLOWED`가 `pages/join.js` 안에 있었습니다	`core/password.js`로 옮기고, `join.js`와 `userinfo.js`가 함께 import합니다

이유: `userinfo.js`가 `join.js`를 import하면 **가입 페이지의 코드(맨 아래의 init 줄들)까지 실행**됩니다. 규칙만 따로 두면 두 화면이 같은 규칙을 쓸 수 있습니다. `join.js`는 import 한 줄만 바뀌었고, 가입 화면 시험(P1-07 16, 17)을 다시 돌려서 모두 통과했습니다. P1-07 16 문서의 "규칙 위치" 줄도 고쳤습니다.

HTML에 더한 것: Change password 버튼 옆에 `<span id="password-saved" role="status">`를 넣었습니다 (24번의 Saved와 같은 모양, 같은 CSS).

3. 코드

import (파일 맨 위)

```
import { api } from '../core/api.js';
import { PASSWORD_RULES, PASSWORD_ALLOWED } from '../core/password.js';
```

`core/password.js` 줄을 이번에 추가했습니다.

`core/password.js` (새 파일이며, `join.js`에 있던 것을 그대로 옮겼습니다)

```
export const PASSWORD_RULES = {
  length: (pw) => pw.length >= 8 && pw.length <= 20,
  upper: (pw) => /[A-Z]/.test(pw),
  lower: (pw) => /[a-z]/.test(pw),
  number: (pw) => /[0-9]/.test(pw),
  special: (pw) => /[!-/:-@[-`{-~]/.test(pw),
};
export const PASSWORD_ALLOWED = /^[!-~]*$/;
```

함수: 준비와 입력할 때

```
export function initPasswordForm() {
  const form = document.getElementById('password-form');
  const current = document.getElementById('pw-current');
  const password = document.getElementById('pw-new');
  const confirm = document.getElementById('pw-confirm');
  const currentField = document.getElementById('pw-current-field');
  const passwordField = document.getElementById('pw-new-field');
  const confirmField = document.getElementById('pw-confirm-field');
  const items = document.querySelectorAll('#pw-new-rules li[data-rule]');
  const error = document.getElementById('password-error');
  const saved = document.getElementById('password-saved');
  const submit = form ? form.querySelector('button[type="submit"]') : null;
  if (!form || !current || !password || !confirm) return;
  let savedTimer = null;

  const allMet = () => {
    if (!PASSWORD_ALLOWED.test(password.value)) return false;
```

```
  for (const name of Object.keys(PASSWORD_RULES)) {
    if (!PASSWORD_RULES[name](password.value)) return false;
  }
  return true;
};

const showRules = () => {
  for (const item of items) {
    const rule = PASSWORD_RULES[item.dataset.rule];
    const met = rule ? rule(password.value) : false;
    item.classList.toggle('is-met', met);
    const state = item.querySelector('[data-rule-state]');
    if (state) state.textContent = met ? ', met' : ', not met';
  }
};

const checkConfirm = (force) => {
  if (confirm.value === '' || confirm.value === password.value) {
    setFieldState(confirmField, null);
  } else if (force || confirm.value.length >= password.value.length) {
    setFieldState(confirmField, 'error', "Passwords don't match.");
  }
};

const clearSaved = () => {
  if (saved) saved.textContent = '';
};

current.addEventListener('input', () => {
  setFieldState(currentField, null);
  clearSaved();
});
password.addEventListener('input', () => {
  showRules();
  clearSaved();
  if (!PASSWORD_ALLOWED.test(password.value)) {
    setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
  } else {
    setFieldState(passwordField, null);
  }
  if (confirm.value !== '') checkConfirm(true);
});
password.addEventListener('blur', () => {
  if (password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet()) {
    setFieldState(passwordField, 'error', 'Check the password requirements.');
  }
});
confirm.addEventListener('input', () => {
  clearSaved();
  checkConfirm(false);
});
confirm.addEventListener('blur', () => checkConfirm(true));
```

함수: 보내기

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    if (error) error.hidden = true;
    const problems = [];
    if (current.value === '') {
      setFieldState(currentField, 'error', 'Enter your current password.');
      problems.push(current);
    }
    if (!allMet()) {
      setFieldState(passwordField, 'error', 'Check the password requirements.');
      problems.push(password);
    }
    if (confirm.value !== password.value) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
      problems.push(confirm);
    }
    if (problems.length > 0) {
      problems[0].focus();
      return;
    }

    if (submit) submit.disabled = true;
    try {
      await api.put('/users/me/password', { current_password: current.value, new_password: password.valu
e });
      current.value = '';
      password.value = '';
      confirm.value = '';
      setFieldState(currentField, null);
      setFieldState(passwordField, null);
      setFieldState(confirmField, null);
      showRules();
      if (saved) {
        saved.textContent = 'Password changed.';
        clearTimeout(savedTimer);
        savedTimer = setTimeout(clearSaved, 5000);
      }
    } catch (err) {
      if (err.code === 'WRONG_PASSWORD') {
        setFieldState(currentField, 'error', 'Current password is incorrect.');
        current.value = '';
        current.focus();
      } else if (err.status === 422 && err.details && err.details.new_password) {
        setFieldState(passwordField, 'error', err.message);
        password.focus();
      } else if (error) {
        error.textContent = err.message || 'Something went wrong. Please try again.';
        error.hidden = false;
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const allMet = () => PASSWORD_ALLOWED.test(password.value)
  && Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
const met = PASSWORD_RULES[item.dataset.rule]?.(password.value) ?? false;
item.querySelector('[data-rule-state]')?.replaceChildren(met ? ', met' : ', not met');
…
} else if (err.status === 422 && err.details?.new_password) {
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of Object.keys(…)` 로 돌다가 하나라도 틀리면 `return false`	모두 참인지 확인합니다
`rule?.(value) ?? false`	`rule ? rule(value) : false`	함수가 있을 때만 부르고, 없으면 `false` 입니다
`el?.replaceChildren(글)`	`if (state) state.textContent = 글`	있을 때만 글자를 바꿉니다
`err.details?.new_password`	`err.details && err.details.new_password`	`details` 가 있을 때만 읽습니다

문법 풀이

import 문법	뜻
`import { PASSWORD_RULES, PASSWORD_ALLOWED } from '../core/password.js'`	새 공용 파일에서 이름 있는 내보내기 두 개를 가져옵니다

식	뜻
`const problems = []`, `problems.push(input)`, `problems[0].focus()`	틀린 칸을 화면 위쪽부터 모아서 첫 칸에 커서를 둡니다 (가입 17번과 같습니다)
`err.code === 'WRONG_PASSWORD'`	서버(P1-09 44)가 400과 함께 주는 코드입니다
`err.details.new_password`	서버 422의 칸 이름입니다. 값은 규칙 목록이거나 `'same_as_current'` 입니다
`setTimeout(clearSaved, 5000)`	5초 뒤에 문구를 지웁니다 (함수를 그대로 넘깁니다)

4. 흐름

입력할 때: 지금 비밀번호 칸은 오류를 지웁니다. 새 비밀번호 칸은 조건 목록과 한글 여부를 검사하고, 확인 칸을 다시 비교합니다. 확인 칸은 다 쳤으면 비교합니다. 어느 칸이든 입력하면 "Password changed."를 지웁니다. **보낼 때:** 세 칸을 검사해서 틀린 칸이 있으면 첫 칸에 커서를 두고 끝냅니다. 맞으면 버튼을 잠그고 PUT을 보냅니다. 성공하면 칸을 비우고 안내를 보여 주고, 400이면 지금 비밀번호 칸에, 422면 새 비밀번호 칸에, 그 밖의 오류는 폼 아래에 보여 준 뒤 잠금을 풉니다.

5. 왜 이렇게

선택	이유
가입 화면(16)과 같은 규칙, 문구, 시점	같은 비밀번호 규칙을 두 화면에서 다르게 보여 주지 않습니다 (조건 미달은 칸을 벗어날 때, 한글은 바로, 확인 칸은 다 친 뒤)

선택	이유
규칙만 공용 파일로 옮기고, 화면 동작 코드는 각자 둠	가입 화면의 16번은 검토가 끝난 코드입니다. 화면 동작까지 묶으려면 16번을 다시 짜야 하므로, 템플릿이 권한 범위(규칙)만 옮겼습니다. 비슷한 화면 코드가 두 곳에 있는 것은 나중에 정리할 수 있습니다
지금 비밀번호가 틀리면 그 칸을 비움	로그인 모달(P1-07 13)과 같습니다. 다시 치게 합니다
성공하면 세 칸을 비움	브라우저에 비밀번호가 남아 있지 않게 합니다. 조건 목록도 ○로 되돌립니다
다시 로그인시키지 않음	서버도 새 토큰을 주지 않습니다 (47 §5). 지금의 로그인은 그대로 유지됩니다
"Password changed."는 5초 (닉네임의 Saved는 3초)	다른 칸들이 비워져서 변화가 크므로, 무엇이 일어났는지 읽을 시간을 조금 더 줍니다

6. 시험 (실제 userInfo.html, 실제 서버, 12개)

시험	확인한 것
`Studio1`	대문자, 소문자, 숫자가 ✓이고, 입력 중에는 문구가 없으며, 칸을 벗어나면 "Check the password requirements."가 나옵니다
한글	바로 "Use only English ..."가 나옵니다
확인 칸	같은 길이인데 다르면 문구가 나오고, 같아지면 지워지며, 조건 5개가 ✓입니다
빈 칸으로 보냄	세 칸에 문구가 나오고, 커서는 지금 비밀번호 칸에 있으며, 요청은 0번입니다
지금 비밀번호가 틀림	보내는 동안 버튼이 잠기고, "Current password is incorrect."가 나오며, 그 칸이 비워지고 커서가 갑니다
새 비밀번호가 지금과 같음	새 칸에 "Use a different password."가 나오고, 커서가 갑니다
바꾸기	세 칸, 조건, 문구가 처음 상태로 돌아가고 "Password changed."가 나옵니다. **실제 로그인 API**로 확인하니 새 비밀번호는 200, 옛 비밀번호는 401이었습니다
되돌리기	원래 비밀번호로 로그인해서 200을 받았습니다

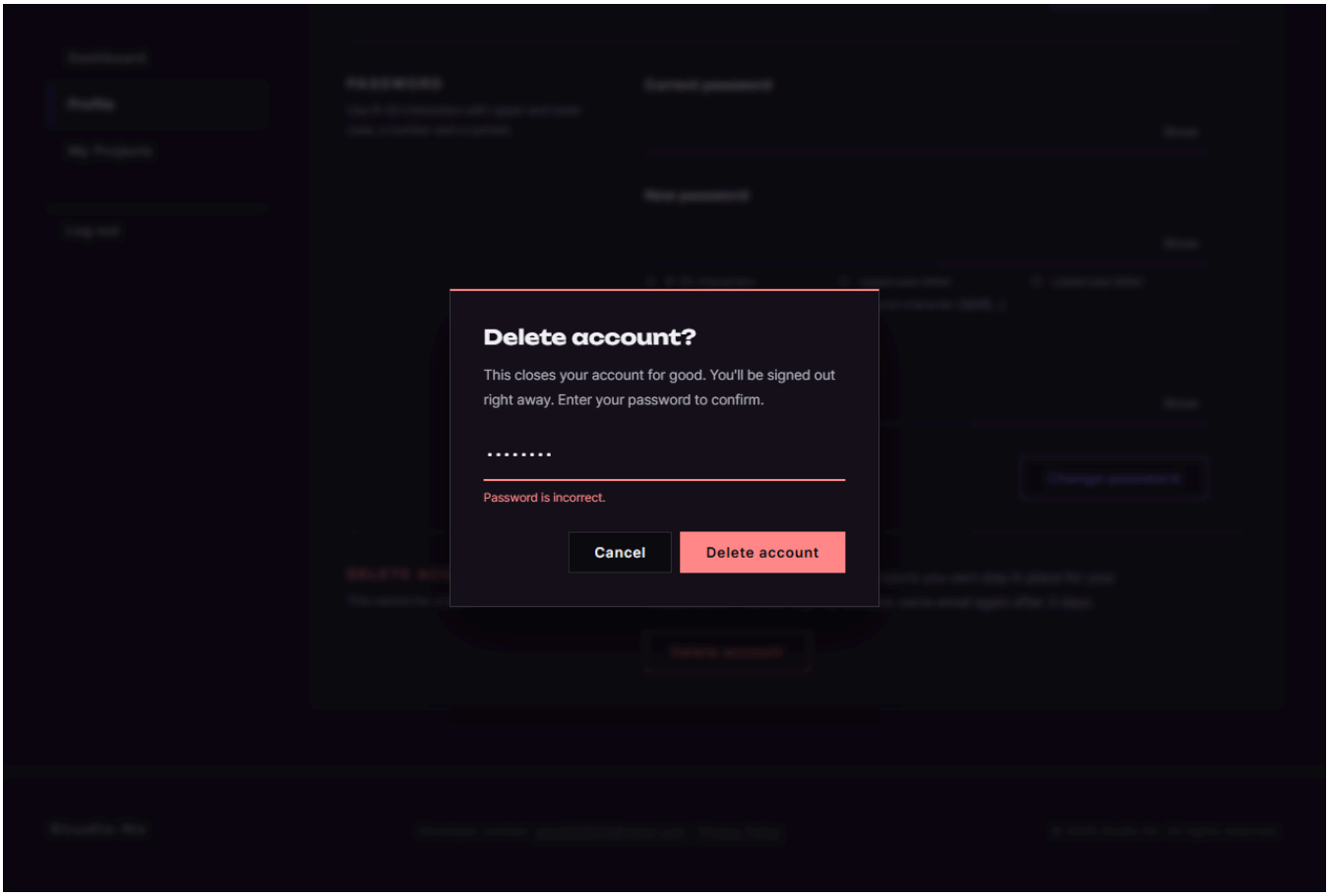
12개 모두 통과했습니다. 22, 23, 24번과 가입 화면(P1-07 16, 17) 시험도 다시 돌려서 모두 통과했습니다 (가입 시험이 만든 계정은 지웠습니다).

JS 26 — `initWithdraw()` (템플릿 `initWithdrawForm`)

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js`, `userInfo.html` (탈퇴 모달 삭제, 버튼), `backend/app/schemas/user.py` (주석 한 줄)
작업	P1-10 회원 정보 화면 JS: 회원 탈퇴 (PRD 01 FR-108)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	25 `initPasswordForm()` · 27 `initLogoutButton()`

1. 하는 일

Delete account를 누르면 FIX-01의 확인창(`promptDialog`)으로 비밀번호를 받고, 맞으면 탈퇴한 뒤 홈으로 보냅니다.



한 일	결과
Delete account	빨간 확인창 "Delete account?"가 뜨고, 비밀번호 칸에 커서가 갑니다
Cancel, Esc, 바깥 클릭	창이 닫히고, 아무 일도 일어나지 않습니다
빈 채로 Delete account	창 안에 "Enter your password."가 나옵니다
틀린 비밀번호	창 안에 "Password is incorrect."가 나옵니다. **창은 그대로 있으므로** 다시 입력할 수 있습니다

한 일	결과
맞는 비밀번호	탈퇴한 뒤 `home.html`로 갑니다 (헤더는 Sign In, Join Us). 뒤로 가기를 눌러도 회원 정보 화면으로 돌아오지 않습니다

2. 언제 쓰나

userInfo.html이 열릴 때 한 번 부릅니다. 이어지는 것들은 다음과 같습니다.

순서	하는 곳
① 확인창, 비밀번호 칸, 확인 중 잠금	FIX-01 21 `promptDialog`
② 탈퇴 요청	백엔드 48 `DELETE /users/me` → 46 (소프트 삭제, 사진 삭제), 리프레시 쿠키 삭제
③ 메모리 토큰 지우기	P1-02 `clearAccessToken`

사용자 결정 반영 (2026-10-07, FIX-01 목업 08, 14)

탈퇴 확인은 **비밀번호만** 받고(체크박스 없음), SweetAlert 확인창으로 하기로 했습니다. 그래서 HTML을 다음과 같이 바꿨습니다.

전	지금
`<dialog id="withdraw-modal">`: 비밀번호 칸, "I understand..." 체크박스, 오류 줄, 버튼	**삭제했습니다** (그 자리에 "확인창은 JS가 띄움"이라는 주석을 남겼습니다)
Delete account 버튼의 `command="show-modal"` `commandfor="withdraw-modal"`	`id="withdraw-button"`으로 바꾸고, JS가 누름을 받습니다
함수 이름 `initWithdrawForm`	더 이상 폼이 아니라서 `initWithdraw`로 바꿨습니다. 아래쪽의 호출 줄과 서버 스키마 주석도 함께 바꿨습니다

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken } from '../core/api.js';
import { promptDialog } from '../core/dialog.js';
```

`clearAccessToken`과 `dialog.js` 줄을 이번에 추가했습니다.

함수

```
export function initWithdraw() {
  const button = document.getElementById('withdraw-button');
  if (!button) return;

  button.addEventListener('click', async () => {
    const password = await promptDialog({
      title: 'Delete account?',
      text: "This closes your account for good. You'll be signed out right away. Enter your password to
confirm.",
      inputType: 'password',
      placeholder: 'Password',
      confirmText: 'Delete account',
```

```
      danger: true,
      requiredMessage: 'Enter your password.',
      check: async (value) => {
        try {
          await api.delete('/users/me', { body: { password: value } });
          return null; // 탈퇴됨 → 창이 닫힘
        } catch (err) {
          return err.code === 'WRONG_PASSWORD' ? 'Password is incorrect.' : err.message;
        }
      },
    });
    if (password === null) return; // 취소 · Esc · 바깥 클릭
    clearAccessToken();
    location.replace('home.html');
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
check: (value) => api.delete('/users/me', { body: { password: value } })
  .then(() => null)
  .catch((err) => (err.code === 'WRONG_PASSWORD' ? 'Password is incorrect.' : err.message)),
…
document.getElementById('withdraw-button')?.addEventListener('click', async () => { … });
```

최신 문법	실제 코드	뜻
`.then(…).catch(…)` 이어 쓰기	`try { await … } catch (err) { … }`	성공과 실패를 나눕니다. `try`가 더 읽기 쉽습니다
`el?.addEventListener(…)`	`if (!button) return;`	버튼이 있을 때만 연결합니다

문법 풀이

import 문법	뜻
`import { api, clearAccessToken } from '../core/api.js'`	같은 파일에서 두 개를 가져옵니다. `clearAccessToken`은 메모리의 Access Token을 지우는 함수입니다 (P1-02)
`import { promptDialog } from '../core/dialog.js'`	FIX-01 21번으로, 입력칸이 하나 있는 확인창입니다

식	뜻
`check: async (value) => { … }`	객체 칸에 함수를 넣습니다. 확인을 누르면 `promptDialog`가 이 함수를 부릅니다. 문자열을 돌려주면 창 안에 빨갛게 보여 주고, `null`을 돌려주면 창을 닫습니다
`api.delete('/users/me', { body: { … } })`	`api.delete`의 두 번째 인자는 **옵션**이므로, 본문은 그 안의 `body`로 넘깁니다 (P1-09 48에서 찾아 고친 점입니다)
`err.code === 'WRONG_PASSWORD'`	서버(P1-09 44)가 400과 함께 주는 코드입니다
`password === null`	확인을 누르지 않고 창이 닫혔다는 뜻입니다 (21번의 약속)
`location.replace('home.html')`	방문 기록에 회원 정보 화면을 남기지 않고 옮겨 갑니다

4. 흐름

1. 버튼을 누르면 확인창을 엽니다.
2. 확인을 누르면, 빈 칸일 때는 창 안에 문구를 보여 주고, 아니면 서버에 탈퇴를 요청합니다 (그동안 취소와 Esc가 잠깁니다).
3. 비밀번호가 틀리면 창 안에 문구를 보여 주고, 계속 입력하게 합니다.
4. 성공하면 창이 닫히고, 메모리 토큰을 지운 뒤 홈으로 갑니다.
5. 취소하면 아무 일도 일어나지 않습니다.

5. 왜 이렇게

선택	이유
확인창 **안에서** 서버에 요청 (`check`)	비밀번호가 틀렸을 때 창을 다시 열 필요가 없습니다. 예전 모달도 틀리면 모달 안에 문구를 보여 주었습니다
체크박스 없음	사용자의 결정입니다. 빨간 선, 빨간 버튼, 설명, 비밀번호 입력이 이미 확인 단계 역할을 합니다
설명에 닉네임을 넣지 않음 ("your account")	템플릿 모달은 닉네임을 보여 주었지만, 닉네임 칸(24)에 저장하지 않은 글자가 있으면 틀린 이름을 보여 줄 수 있습니다
성공하면 `clearAccessToken`	서버가 쿠키는 지웠지만, 이 페이지의 메모리에는 Access Token(최대 30분)이 남아 있습니다. 어차피 서버가 막지만(06), 깨끗이 지웁니다
`location.replace` (템플릿은 `location.href`)	뒤로 가기를 누르면 탈퇴한 계정의 회원 정보 화면으로 돌아가서 오류가 납니다
오류 문구는 서버 문구를 그대로 쓰되, 비밀번호 틀림만 따로 정함	서버 문구 "Password is incorrect."와 같지만, 화면 문구를 이 함수가 정하도록 코드로 확인합니다

6. 시험 (실제 userInfo.html, 실제 서버, 10개)

탈퇴하면 그 계정을 다시 쓸 수 없으므로, 이 시험만 따로 만든 계정(`sitetest.p110w@example.com`)으로 하고, 끝난 뒤 지웠습니다.

시험	확인한 것
HTML	`#withdraw-modal`이 없고, 버튼에 `command`가 없습니다
열기	제목은 "Delete account?"이고, 비밀번호 칸에 커서가 있으며, 빨간 창과 빨간 "Delete account" 버튼이 나옵니다
Cancel	창이 닫히고, 로그인은 그대로이며, 탈퇴되지 않고, 주소도 그대로입니다
빈 칸, 틀린 비밀번호	"Enter your password.", "Password is incorrect."가 나오고(창은 그대로), 탈퇴되지 않습니다
맞는 비밀번호 (옮겨 간 뒤 확인)	`/home.html`로 가고, `isLoggedIn()`은 false이며, 헤더는 로그아웃 상태입니다. 그 계정으로 로그인하면 401이고, 방문 기록 수는 그대로입니다 (replace)

처음에는 9/10이었습니다. 방문 기록 수를 "2"로 고정해서 비교한 **시험 쪽의 실수**였습니다 (시험 도구가 먼저 연 빈 페이지와 서버 주소도 기록에 들어갑니다). 누르기 전후의 수가 같은지 비교하도록 고쳐서 10개 모두 통과했습니다. 22~25번 시험도 다시 돌려서 모두 통과했습니다.

JS 27 — `initLogoutButton()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js` (새 함수), `pages/userinfo.js` (import), `pages/mypage.js`, `pages/myproject.js` (주석)
작업	P1-10 회원 정보 화면 JS: 왼쪽 메뉴의 Log out (PRD 01 FR-105)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	26 `initWithdraw()` · 28 `initContactForm()` (P1-12)

1. 하는 일

왼쪽 메뉴의 **Log out**을 누르면 버튼을 잠그고 로그아웃한 뒤 홈으로 갑니다.

누른 뒤	결과
바로	버튼이 잠깁니다 (두 번 눌러도 요청은 한 번만 갑니다)
서버	`POST /api/v1/auth/logout`을 받고 리프레시 쿠키를 지웁니다 (P1-05 34)
화면	메모리 토큰을 지우고 `home.html`로 갑니다 (헤더는 Sign In, Join Us). 뒤로 가기를 눌러도 돌아오지 않습니다

실제 일은 이미 있는 P1-02 08 `logout()`이 합니다. 이 함수는 버튼에 연결하고 잠그는 일만 합니다.

2. 언제 쓰나

화면	버튼	지금 상태
userInfo.html	`#logout-button`	✅ 이번에 연결했습니다 (`userinfo.js` 맨 아래의 `initLogoutButton();`)
myPage.html	같은 `#logout-button`	P1-17 때 import만 하면 됩니다 (`mypage.js` 템플릿 주석에 적어 두었습니다)
myProject.html	같은 `#logout-button`	P1-14 때 import만 하면 됩니다 (`myproject.js` 주석)

세 화면이 공통으로 쓰므로 `core/auth-ui.js`에 하나만 두었습니다. 템플릿은 페이지마다 같은 함수를 하나씩 두었습니다(`userinfo.js`, `mypage.js`, `myproject.js`). `userinfo.js`의 빈 함수는 지우고 import로 바꿨습니다. 나머지 두 페이지의 빈 함수는 해당 작업(P1-14, P1-17) 때 지웁니다.

3. 코드

import (`auth-ui.js` 맨 위)

```
import { api, setAccessToken } from './api.js';
import { isLoggedIn, parseLoginHash, logout } from './auth.js';
```

`logout`을 이번에 추가했습니다. `userinfo.js`는 `import { …, initLogoutButton } from '../core/auth-ui.js';`로 바꾸고, 쓰지 않게 된 `logout` import를 뺐습니다.

함수

```
export function initLogoutButton(button = document.getElementById('logout-button')) {
  if (!button || button.dataset.logout === 'on') return;
  button.dataset.logout = 'on';
  button.addEventListener('click', () => {
    button.disabled = true;
    logout(); // 끝나면 다른 페이지로 옮겨 가므로 다시 켤 필요 없음
  });
}
```

최신 문법으로 쓰면 (공부용)

```
export function initLogoutButton(button = document.getElementById('logout-button')) {
  if (button?.dataset.logout === 'on' || !button) return;
  button.dataset.logout = 'on';
  button.addEventListener('click', () => { button.disabled = true; logout(); }, { once: true });
}
```

최신 문법	실제 코드	뜻
`button?.dataset …`	`!button || …` (먼저 없는지 확인합니다)	버튼이 있을 때만 처리합니다
`{ once: true }`	`button.disabled = true`로 막음	한 번만 받습니다. 잠긴 상태가 화면에도 보이는 쪽을 택했습니다

문법 풀이

식	뜻
`(button = document.getElementById('logout-button'))`	**매개변수 기본값**입니다. 넘기지 않으면 `#logout-button`을 쓰므로, 다른 버튼에도 쓸 수 있습니다
`button.dataset.logout === 'on'`	이미 연결했는지 확인합니다 (P1-07 10번 비밀번호 토글과 같은 방법)
`logout();` (`await` 없음)	기다릴 필요가 없습니다. 끝나면 페이지가 바뀌기 때문입니다

4. 흐름

1. 버튼이 없거나 이미 연결했으면 끝냅니다.
2. 누르면 버튼을 잠그고 `logout()`을 부릅니다 (서버 쿠키 삭제 → 메모리 토큰 삭제 → 홈으로 이동).

5. 왜 이렇게

선택	이유
공용 파일에 하나만 둠	세 화면의 버튼과 동작이 똑같습니다. 템플릿처럼 세 번 쓰면 고칠 곳도 세 곳이 됩니다
`auth-ui.js`에 둠	로그인, 헤더 상태 등 "인증 화면 동작"을 모아 둔 곳입니다 (P1-07)
버튼 잠금	느린 망에서 여러 번 눌러 로그아웃 요청이 여러 번 가지 않게 합니다
두 번 연결 방지	페이지가 실수로 두 번 불러도 요청은 한 번만 갑니다 (시험으로 확인했습니다)
확인창 없음	로그아웃은 다시 로그인하면 되돌릴 수 있습니다. 확인창은 되돌릴 수 없는 일(탈퇴, 삭제)에만 씁니다

6. 시험 (실제 userInfo.html, 실제 서버, 5개)

시험	확인한 것
연결	`data-logout="on"`이 붙고, 한 번 더 불러도 연결이 늘어나지 않습니다
누름	바로 버튼이 잠기고, 로그아웃 요청은 **1번**입니다
옮겨 간 뒤	`/home.html`이고, 방문 기록 수는 그대로이며, `isLoggedIn()`은 false이고, 헤더는 로그아웃 상태입니다

5개 모두 통과했습니다. `auth-ui.js`를 고쳤으므로 그 파일을 쓰는 P1-07 시험(09, 10, 12, 13)과 P1-10 시험(22~25)도 다시 돌려서 모두 통과했습니다 (시험이 만든 계정은 지웠습니다).

처음 한 번은 **시험 쪽 문제**로 결과를 받지 못했습니다. 로그아웃이 너무 빨라서 시험 코드가 끝나기 전에 페이지가 바뀌었기 때문입니다. 요청 수를 `sessionStorage`에 바로 적고, 누른 직후에 끝내도록 고쳤습니다.